

R2.01 - Développement Orienté Objets

Une premiere classe Kotlin : la classe `String`

Arnaud Lanoix Brauer
Arnaud.Lanoix@univ-nantes.fr



Nantes Université

Département informatique

La classe `String`

- Sans le savoir, vous manipulez déjà des objets en Kotlin :

La classe `String` par exemple

- Représentation interne "complexe" = un tableau de `Char` + ... :

- On n'a pas directement accès à la structure interne ; elle est **privée**

▶ `var chaine = "ToToRo"` $\equiv$

0	1	2	3	4	5
'T'	'o'	'T'	'o'	'R'	'o'

▶ `chaine` est un objet, c'est une instance de `String`

- La classe `String` hérite de `CharSequence` et de `Comparable<String>`

- La documentation complète :

<https://kotlinlang.org/api/latest/jvm/stdlib/kotlin/-string/>

La classe `String` (2)

`var chaine = "ToToRo"` $\equiv$

0	1	2	3	4	5
'T'	'o'	'T'	'o'	'R'	'o'

- propriétés

- ▶ `length` – longueur de la chaîne
- ▶ `lastIndex` – indice du dernier caractère
- ▶ ...

- méthodes

- ▶ `isEmpty() : Boolean` – la chaîne est vide
- ▶ `uppercase() : String` – en majuscule
- ▶ `lowercase() : String` – en minuscule
- ▶ `reversed() : String` – renversé

```
val longueur = chaine.length
var indice = chaine.lastIndex

var verif : Boolean
verif = chaine.isEmpty()
chaine2 = chaine.uppercase()
chaine2 = chaine.lowercase()
chaine2 = chaine.reversed()
```

Les appels aux propriétés et aux méthodes d'un objet se font avec un `.` comme dans `chaine.length` ou `chaine.isEmpty()`

La classe `String` (3)

```
var chaine = "ToToRo" ≡
```

0	1	2	3	4	5
'T'	'o'	'T'	'o'	'R'	'o'

- Accès indicé

- ▶ `get(i: Int): Char` ≡

- ▶ notation `[i]`
(comme pour un tableau)

- Concaténation

- ▶ `plus(other: String) : String` ≡

- ▶ opérateur `+`

- Egalité

- ▶ `equals(other: String) : Boolean`

≡ opérateurs `==` et `!=`

- sous-chaînes

- ▶

```
substring(start: Int, end: Int) : String
```

```
var carac = chaine.get(0)
carac = chaine.get(0)
carac = chaine[0]
carac = chaine[3]

var chaine2 : String
chaine2 = "Tonari no ".plus(chaine)
chaine2 = "Tonari no " + chaine

verif = chaine.equals("ToToRo")
verif = chaine == "ToToRo"
verif = chaine != "totoro"

chaine2 = chaine.substring(2, 4)
chaine2 = chaine.substring(2)
```

Et les autres types de base ?

Tous les types de base `Int`, `Double`, `Boolean`, `Char`, etc. sont des **classes**. Ils offrent certaines méthodes et propriétés.

Le type `Int`

- `toDouble() : Double`
- `toString() : String`
- `plus(other : Int) : Int` $\equiv$ `+`, `times(other : Int) : Int` $\equiv$ `*`,
...
- <https://kotlinlang.org/api/latest/jvm/stdlib/kotlin/-int/>

Les tableaux `Array<X>` sont aussi des classes.